

EJERCICIO 1) 12 PUNTOS

Se desea administrar perros de un refugio.

Las operaciones son las siguientes:

1) Nuevo perro

Cuando ingresa un nuevo perro al refugio, se le asigna un código único alfanumérico, se le da un nombre, se lo asigna a una casilla para que duerma (identificada con un número natural), se guarda la edad y se lo analiza para determinar si tiene heridas y asignarle una prioridad (número entre 1 y 30).

```
void agregarPerro(string codigo, string nombre, int casilla, int edad, int prioridad);
```

Esta operación debe realizarse en **$O(\log n)$** en el peor caso, siendo n la cantidad de perros del refugio.

2) Obtener casilla

Dado el nombre de un perro, se quiere obtener el identificador de la casilla en la que se encuentra.

```
int obtenerCasilla(string nombrePerro);
```

Esta operación debe realizarse en **$O(1)$** en el caso promedio.

3) Obtener perro herido más prioritario.

Se quiere obtener el nombre del perro herido más prioritario.

```
string obtenerPerroHeridoMasPrioritario();
```

Esta operación debe realizarse en **$O(1)$** en el peor caso.

4) Perro curado.

Una vez que un perro es atendido, se lo debe de dejar de tomar en cuenta para la operación 3).

`void perroMasPrioritarioFueCurado();`

Esta operación debe realizarse en $O(\log n)$ peor caso, siendo n la cantidad de perros heridos.

5) Perros por edad

Para poder tener una mejor organización, se debe poder listar los perros de cierta edad, ordenados por nombre.

`void listarPerrosDeEdad(int edad);`

Esta operación debe poder realizarse en $O(n)$ en el peor caso, siendo n la cantidad de perros de dicha edad.

Se sabe que, lamentablemente, la mayor esperanza de vida de un perro es la de un perro pequeño y se aproxima en los 20 años.

Se pide:

- a) Elegir las estructuras adecuadas para resolver las operaciones (indicar tipos). Realizar un boceto de la solución y justificar las cotas temporales.
- b) Implementar la operación 2) y 3).

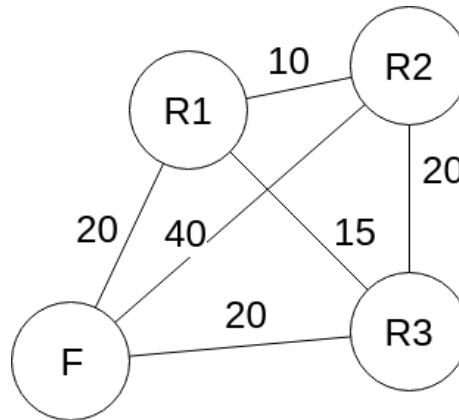
EJERCICIO 2) 13 PUNTOS

Una persona decide instalar un sistema de riego con regadores por zonas. Ya instaló los regadores según las zonas a regar, aún sin conexión a una fuente de agua.

Se pueden conectar directamente todos los regadores entre sí y con la fuente de agua, conociendo las distancias en metros.

Para representar esta situación, decide utilizar un grafo de tal forma que los vértices son los regadores y la fuente, y las aristas son las posibles conexiones, de las cuales conoce la distancia en metros.

Por ejemplo, si fueran 3 regadores, el grafo planteado podría ser el siguiente:



1. ¿Qué tipo de grafo es el que plantea? Describa las características que crea necesarias.
2. Sabiendo que son R regadores y 1 fuente de agua corriente, ¿cuántas conexiones directas posibles (aristas) entre pares de regadores/fuente (vértices) hay? ¿Cuál es su razonamiento?

En el ejemplo, si son 3 regadores y 1 fuente, hay 6 conexiones posibles.

(F, R1, 20), (F, R2, 40), (F, R3, 20), (R1, R2, 10), (R1, R3, 15) y (R2, R3, 20)

3. Decide armar las conexiones para llevar agua, pero no está seguro cuáles poner. El metro de caño de agua tiene un costo alto y desea utilizar la menor cantidad de metros posible para que todos los regadores tengan agua.

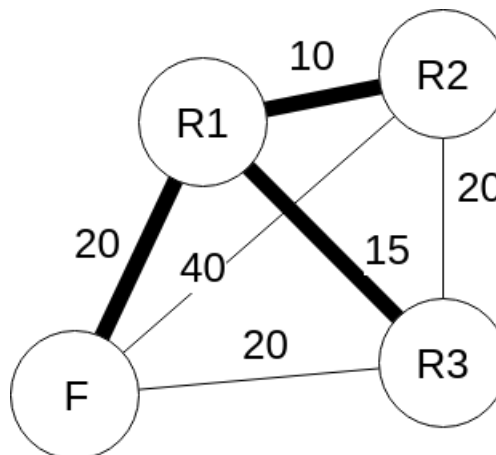
Implemente una operación que retorne un listado de las conexiones que deben instalarse.

Recibe por parámetro:

- Todas las conexiones posibles: conjunto/arreglo/lista de las tripletas. Note que la cantidad de elementos de esta estructura, es la respuesta del punto anterior.
- La cantidad de regadores.

Retorna conjunto/arreglo/lista de las tripletas que representan las conexiones a instalar.

En el ejemplo anterior, una solución podría ser la que contiene las conexiones que en la imagen tienen más grosor:



(F, R1, 20), (R1, R3, 15), (R1, R2, 10). El costo total de metros es 45.

4. Indique el orden del tiempo de ejecución del algoritmo que implementó en el punto anterior.